

Guia paso a paso - Semana 07 ## Caso: Analisis de emprendimientos costarricenses

****Archivo base del estudiante:**** `clase07\_analisis\_emprendimientos.py` ****Fuente de datos:**** `sedes.py` ****Solucion docente de referencia:**** `analisis\_emprendimientos\_solucion.py` > No modifique `sedes.py` ni `clase07\_analisis\_emprendimientos.py`. La guia explica como llevar el archivo de clase desde el import inicial hasta la solucion esperada. --- ## 1. Meta de la sesion

La red ****EmprendeCR**** tiene varias sedes en Costa Rica. Cada sede registra ventas de lunes a viernes y una meta semanal. Al terminar, el programa debe construir un reporte que indique:

1. Total semanal vendido por sede.
2. Promedio diario de ventas.
3. Porcentaje de cumplimiento de la meta.
4. Clasificacion con condicionales.
5. Provincias analizadas sin repetir.
6. Ranking base con nombre de sede y total.
7. Sede o sedes con la venta mas alta, incluyendo empates.

La clase no busca solo imprimir datos. Busca practicar como un algoritmo convierte una lista de registros en informacion util para decidir. --- ## 2. Recurso inicial: lista, tupla, diccionario o set

Antes de programar, defina que estructura responde mejor a cada necesidad.

- | Estructura | Se escribe | Cuando usarla | Ejemplo en EmprendeCR | Cuidado principal | |---|---|---|---| | Lista | `[]` | Cuando hay varios datos relacionados y conviene recorrerlos o conservar su orden. | Ventas de lunes a viernes: `[85000, 92000, 78000]` | No depender de indices sin explicar que representan. | | Tupla | `()` | Cuando quiero guardar un par o grupo fijo que no necesito modificar. | Par para ranking: `("Soda San Pedro", 462000)` | No intentar cambiar sus valores como si fuera lista. | | Diccionario | `{}` | Cuando un registro tiene campos con nombre. | Una sede: `{"nombre": "Soda San Pedro", "ventas": [...]}` | Escribir exactamente las claves. | | Set | `set()` | Cuando necesito valores unicos, sin repetidos. | Provincias presentes: `{"San Jose", "Cartago"}` | No usarlo cuando el orden importa. |

Regla rapida de decision - Use ****lista**** si son varios valores que se recorren. - Use ****diccionario**** si cada dato tiene nombre propio. - Use ****tupla**** si necesita guardar un par fijo de resultado. - Use ****set**** si necesita eliminar repetidos automaticamente.

Como se aplica en este caso | Necesidad del programa | Estructura recomendada | Por que | |---|---| | Guardar todas las sedes | Lista de diccionarios | Permite hacer `for sede in sedes`. | | Guardar los datos de una sede | Diccionario | Cada campo tiene nombre: `nombre`, `provincia`, `ventas`, `meta`. | | Guardar las ventas diarias | Lista | Son varios numeros relacionados. | | Guardar provincias sin repetir | Set | Evita duplicados sin escribir condicionales extra. | | Guardar nombre y total para ranking | Tupla | Es un par fijo: `(nombre, total)`. | | Guardar el reporte final | Lista de diccionarios | Cada fila del reporte tiene varios campos calculados. |

El recurso visual tambien queda disponible en `recurso_estructuras_datos_semana07.html`. --- ## 3. Punto de partida

El archivo base inicia asi: `python """Practica Semana 07: analisis de emprendimientos costarricenses. Complete los espacios marcados con TODO. El objetivo es generar un reporte por sede usando listas, diccionarios, funciones, ciclos y condicionales. """ from sedes import sedes``

Por que iniciar con `from sedes import sedes`` `sedes.py` contiene los datos. El archivo de clase contiene el algoritmo. Esta separacion permite concentrarse en procesar informacion sin copiar datos dentro del programa principal. --- ## 4. Explorar los datos antes de calcular

Debajo del import, puede usar temporalmente estas lineas para explorar: `python #print("Cantidad de sedes:", len(sedes)) #print("Tipo de variable sedes:", type(sedes)) #print("Tipo de variable sedes[0]:", type(sedes[0])) #print("Datos por sede:", sedes[0].keys()) #print("Primera sede:", sedes[0]["nombre"])``

Cuando el programa final ya este armado, deje estas lineas comentadas para que la salida coincida con el reporte esperado. #### Por que hacerlo

Antes de escribir ciclos y funciones, debe confirmar que el import funciona y que entiende la forma de los datos. El algoritmo depende de claves como `ventas`, `meta`, `nombre`, `provincia` y `tipo`. --- ## 5. Funcion `calcular\_total`

Agregue la primera funcion debajo del import y antes del codigo principal: `python def calcular_total(ventas): """Recibo una lista, la sumo y retorno el total.""" return sum(ventas)``

Para que sirve

Recibe la lista de ventas de una sede y retorna el total semanal. Es una funcion pequena, pero evita repetir `sum(ventas)` en varias partes del programa. #### Ejemplo mental

Si una sede tiene ventas `[85000, 92000, 78000, 110000, 97000]`, el total debe ser `462000`. --- ## 6. Funcion `calcular\_promedio`

Agregue la funcion de promedio: `python def calcular_promedio(lista): """Retorna el promedio de ventas de una lista.""" return sum(lista) / len(lista)``

Por que se usa `len(lista)`

Porque el programa no deberia depender de que siempre haya exactamente cinco dias. Si luego se agregan mas ventas, el promedio sigue funcionando. #### Para que sirve

El promedio diario complementa el

total. Una sede puede cumplir la meta, pero el promedio ayuda a explicar el ritmo de ventas. ---

7. Funcion `calcular\_porcentaje` Agregue la funcion para calcular el cumplimiento: ``python

```
def calcular_porcentaje(total, meta): """Calcula el porcentaje de cumplimiento de una meta."""
return total / meta * 100 ``
```

Por que retorna un numero y no un texto El porcentaje se usa dos veces: 1. Para decidir la clasificacion con condicionales. 2. Para imprimirlo con formato en el reporte. Por eso conviene retornarlo como numero. El formato `:.2f%` se aplica al imprimir. >

Si durante la clase alguien agrego un parametro `formato=True`, puede conservarlo solo si la funcion sigue permitiendo obtener el porcentaje numerico para comparar. En esta solucion docente se usa la version simple y numerica. ---

8. Funcion `calcular\_clasificacion` Agregue la funcion que convierte el porcentaje en una decision: ``python

```
def calcular_clasificacion(total, meta): """Clasifica la sede segun el porcentaje de cumplimiento de la meta."""
porcentaje = calcular_porcentaje(total, meta)
if porcentaje >= 100: mensaje_sede = "Meta alcanzada."
elif porcentaje >= 80: mensaje_sede = "Meta casi alcanzada, prestar atención."
else: mensaje_sede = "Meta no alcanzada URGE ATENCION."
return mensaje_sede ``
```

Por que esta parte es importante Aqui aparece la ejecucion condicional. El programa toma una ruta distinta segun el porcentaje calculado.

Condicion	Resultado
`porcentaje >= 100`	Meta alcanzada.
`porcentaje >= 80`	Meta casi alcanzada, prestar atención.
Menor que 80	Meta no alcanzada URGE ATENCION.

El orden importa: primero se pregunta por 100 o mas. Si se preguntara primero por 80, una sede con 105 tambien entraria ahi. ---

9. Funcion `imprimir\_reporte` Agregue una funcion para mostrar el reporte de forma ordenada: ``python

```
def imprimir_reporte(datos_reporte): """Imprime el reporte final de ventas por sede."""
print(" REPORTE FINAL")
print("-" * 60)
for fila in datos_reporte:
    print(f"Sede: {fila['nombre']}")
    print(f"Provincia: {fila['provincia']}")
    print(f"Tipo: {fila['tipo']}")
    print(f"Total semanal: {fila['total']:.0f}")
    print(f"Promedio diario: {fila['promedio']:.0f}")
    print(f"Cumplimiento: {fila['porcentaje']:.2f}%")
    print(f"Estado: {fila['estado']}")
print("-" * 60)
print("Cantidad de sedes:", len(datos_reporte))
```

Para que sirve Esta funcion separa la presentacion del calculo. Primero construimos datos limpios en `reporte`; despues imprimimos esos datos.

Que estructura recibe Recibe una **lista de diccionarios**. Cada diccionario representa una fila del reporte. ---

10. Preparar las estructuras del programa principal Despues de las funciones, cree las estructuras que se van a llenar con el ciclo: ``python

```
reporte = []
provincias = set()
ranking = []
venta_mas_alta = 0
sedes_mas_ingresos = []
```

Que guarda cada variable

Variable	Tipo	Para que sirve
`reporte`	Lista	Guardar una fila procesada por cada sede.
`provincias`	Set	Guardar provincias sin repetir.
`ranking`	Lista	Guardar tuplas `(nombre, total)` para comparar sedes.
`venta_mas_alta`	Numero	Recordar el total mas alto encontrado.
`sedes_mas_ingresos`	Lista	Guardar una o varias sedes si hay empate.

11. Recorrer las sedes con un ciclo Agregue el ciclo principal: ``python

```
for sede in sedes:
    ventas = sede["ventas"]
    meta = sede["meta"]
    total_sede = calcular_total(ventas)
    promedio_sede = calcular_promedio(ventas)
    porcentaje_sede = calcular_porcentaje(total_sede, meta)
    estado = calcular_clasificacion(total_sede, meta)
```

Por que usar `for sede in sedes` Porque `sedes` es una lista. Cada vuelta del ciclo procesa un diccionario diferente.

Que pasa dentro del ciclo

1. Se extraen los datos base de la sede actual.
2. Se llaman funciones para calcular resultados.
3. Se guardan resultados para el reporte y resumen final.

12. Agregar cada fila al reporte Dentro del ciclo, despues de calcular las variables, agregue: ``python

```
reporte.append( { "nombre": sede["nombre"], "provincia": sede["provincia"], "tipo": sede["tipo"],
"total": total_sede, "promedio": promedio_sede, "porcentaje": porcentaje_sede, "estado": estado, } )
```

Por que usar un diccionario Porque una fila del reporte tiene varios campos con nombre. Esto hace que `imprimir\_reporte` pueda leer `fila['nombre']`, `fila['total']`, `fila['estado']`, etc. ---

13. Guardar provincias y ranking Tambien dentro del ciclo, agregue: ``python

```
provincias.add(sede["provincia"])
ranking.append((sede["nombre"], total_sede))
```

Por que usar `set` para provincias Si dos sedes fueran de la misma provincia, el set la guardaria una sola vez. No hace falta escribir un `if` para evitar repetidos.

Por que usar tupla en ranking `(sede["nombre"], total\_sede)` es un par fijo: nombre y total. No necesitamos modificar ese par despues de crearlo. ---

14. Conservar una o varias sedes con mayor ingreso Dentro del ciclo, agregue la logica de maximo: ``python

```
if total_sede > venta_mas_alta:
    venta_mas_alta = total_sede
    sedes_mas_ingresos = [sede["nombre"]]
elif total_sede == venta_mas_alta:
    sedes_mas_ingresos.append(sede["nombre"])
```

Por que no usar solo `<=` Si se usa solo `<=`, se puede perder informacion o mezclar casos distintos. Aqui se

separan dos situaciones: | Caso | Accion | |---|---| | La sede supera el maximo anterior | Se actualiza el maximo y se reinicia la lista de ganadoras. | | La sede empata el maximo actual | Se agrega como otra ganadora. | | La sede queda por debajo | No se cambia nada. | Esta parte responde al caso real de empate: si dos sedes tienen la venta mas alta, ambas deben conservarse. --- ## 15. Imprimir el reporte y el resumen final Despues del ciclo, fuera del `for`, agregue: ``python imprimir\_reporte(reporte) print(" RESUMEN FINAL") print("Provincias analizadas:", provincias) print("Ranking base:", ranking) print(f"Venta más alta: ₡ {venta\_mas\_alta:,.0f}") print("Sedes con más ingresos:", \*sedes\_mas\_ingresos) `` #### Por que va fuera del ciclo Si se imprime dentro del ciclo, el reporte aparece incompleto varias veces. Fuera del ciclo se imprime una sola vez, cuando todas las sedes ya fueron procesadas. --- ## 16. Salida esperada aproximada Al ejecutar el programa, debe verse una salida similar a esta: ``text REPORTE FINAL ----- Sede: Soda San Pedro Provincia: San Jose Tipo: Alimentacion Total semanal: ₡462,000 Promedio diario: ₡92,400 Cumplimiento: 102.67% Estado: Meta alcanzada. ----- Sede: Panaderia Cartago Provincia: Cartago Tipo: Panaderia Total semanal: ₡351,000 Promedio diario: ₡70,200 Cumplimiento: 92.37% Estado: Meta casi alcanzada, prestar atención. ----- Sede: Cafeteria Liberia Provincia: Guanacaste Tipo: Cafeteria Total semanal: ₡537,000 Promedio diario: ₡107,400 Cumplimiento: 107.40% Estado: Meta alcanzada. ----- Sede: Feria de Heredia Provincia: Heredia Tipo: Feria Total semanal: ₡380,000 Promedio diario: ₡76,000 Cumplimiento: 97.44% Estado: Meta casi alcanzada, prestar atención. ----- Sede: Marisqueria Puntarenas Provincia: Puntarenas Tipo: Restaurante Total semanal: ₡574,000 Promedio diario: ₡114,800 Cumplimiento: 102.50% Estado: Meta alcanzada. ----- Cantidad de sedes: 5 RESUMEN FINAL Provincias analizadas: {'San Jose', 'Cartago', 'Guanacaste', 'Heredia', 'Puntarenas'} Ranking base: [('Soda San Pedro', 462000), ('Panaderia Cartago', 351000), ('Cafeteria Liberia', 537000), ('Feria de Heredia', 380000), ('Marisqueria Puntarenas', 574000)] Venta más alta: ₡574,000 Sedes con más ingresos: Marisqueria Puntarenas `` > Nota: el orden de `provincias` puede variar porque es un `set`. Eso es normal. --- ## 17. Checklist de revision para estudiantes Antes de entregar o cerrar la practica, revise: - `sedes.py` no fue modificado. - El archivo inicia con `from sedes import sedes`. - Las funciones retornan valores; no imprimen directamente excepto `imprimir\_reporte`. - `calcular\_porcentaje` retorna un numero. - La clasificacion usa `if`, `elif` y `else`. - El ciclo `for sede in sedes` procesa todas las sedes. - `reporte` es una lista de diccionarios. - `provincias` es un set. - `ranking` guarda tuplas `(nombre, total)`. - La logica de mayor venta conserva empates. - El reporte se imprime despues de terminar el ciclo. --- ## 18. Reto corto de cierre Modifique temporalmente los datos para que dos sedes tengan el mismo total mas alto. Ejecute el programa y confirme que `Sedes con más ingresos` muestra ambos nombres. Despues de probar, restaure los datos originales de `sedes.py`.